



Ecole nationale des
Sciences Appliquées – Fès



Compte Rendu:

TP4 : Évaluation et Sélection du Modèle

Réaliser par :

❖ Abir Majdi

Encadré par :

Prof. AKKAD

Année universitaire : 2025/2026

Filière : GDNC2

Introduction

Ce TP a pour objectif de maîtriser les différentes techniques d'évaluation et de sélection de modèles en apprentissage automatique. Nous travaillons sur deux jeux de données issus des TPs précédents :

Dataset	Origine	Usage dans ce TP
Iris	iris.csv – TP2 KNN	Parties 1 & 2 : classification, régression
CustomerDataSet	CustomerDataSet.xls – TP3	Partie 3 : évaluation sensible aux coûts

Partie 1 – Métriques de Classification (KNN / Iris)

```
# Cherche iris.csv dans le même dossier que ce script, ou dans les sous-dossiers courants
script_dir = os.path.dirname(os.path.abspath(__file__))
iris_candidates = [
    os.path.join(script_dir, 'iris.csv'),
    os.path.join(script_dir, 'TP 2 KNN', 'iris.csv'),
    'iris.csv',
]
iris_path = next((p for p in iris_candidates if os.path.exists(p)), None)
if iris_path:
    df_iris = pd.read_csv(iris_path)
    feature_cols = ['SepalLength[cm]', 'SepalWidth[cm]', 'PetalLength[cm]', 'PetalWidth[cm]']
    X_iris = df_iris[feature_cols].values
    y_raw = df_iris['Species'].astype(str).values
    le = LabelEncoder()
    y_iris = le.fit_transform(y_raw)
    class_names = le.classes_ # array de str propres
    feature_names = feature_cols
else:
    # Fallback sklearn si iris.csv introuvable
    from sklearn.datasets import load_iris as _load_iris
    _iris = _load_iris()
    X_iris = _iris.data
    y_iris = _iris.target
    class_names = np.array([str(c) for c in _iris.target_names])
    feature_names = _iris.feature_names
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_iris)
print("=" * 65)
print(" TP4 □ ÉVALUATION ET SÉLECTION DU MODÈLE")
print("=" * 65)
print(f"\nDataset Iris : {X_iris.shape[0]} exemples, {X_iris.shape[1]} features")
print(f"Classes : {[str(c) for c in class_names]}\n")
```

```
Dataset Iris : 150 exemples, 4 features
Classes : ['setosa', 'versicolor', 'virginica']
```

Données : iris.csv (150 exemples, 4 features, 3 classes). Classificateur : KNN (k=5), features normalisées (StandardScaler).

1.1 – Holdout (70/30, sans stratification)

Le Holdout consiste à diviser aléatoirement le dataset en un ensemble d'entraînement (70%) et un ensemble de test (30%).

```
print("\n— 1.1 Holdout (70/30, sans stratification) —")

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_iris, test_size=0.30, random_state=42, shuffle=True
)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

acc_holdout = accuracy_score(y_test, y_pred)
print(f" Taille ensemble d'entraînement : {len(X_train)}")
print(f" Taille ensemble de test : {len(X_test)}")
print(f" Accuracy (Holdout) : {acc_holdout * 100:.2f}%")
```

```
— 1.1 Holdout (70/30, sans stratification) —
 Taille ensemble d'entraînement : 105
 Taille ensemble de test : 45
 Accuracy (Holdout) : 100.00%
```

Paramètre	Valeur	Résultat
Taille entraînement	105 exemples (70%)	
Taille test	45 exemples (30%)	
Accuracy (Holdout)	100.00%	Résultat optimiste sans stratification

1.2 – Holdout Stratifié (70/30, stratify=y)

La stratification garantit que la proportion de chaque classe est identique dans train et test. Cela évite les biais de distribution.

```
print("\n— 1.2 Holdout Stratifié (70/30, stratify=y) —")

X_tr_s, X_te_s, y_tr_s, y_te_s = train_test_split(
    X_scaled, y_iris, test_size=0.30, random_state=42, stratify=y_iris
)

knn_s = KNeighborsClassifier(n_neighbors=5)
knn_s.fit(X_tr_s, y_tr_s)
acc_strat = accuracy_score(y_te_s, knn_s.predict(X_te_s))

# Distribution des classes
unique, counts_all = np.unique(y_iris, return_counts=True)
unique, counts_train = np.unique(y_tr_s, return_counts=True)
unique, counts_test = np.unique(y_te_s, return_counts=True)

print(f" Distribution (total) : {dict(zip([str(c) for c in class_names], counts_all.tolist()))}")
print(f" Distribution (train) : {dict(zip([str(c) for c in class_names], counts_train.tolist()))}")
print(f" Distribution (test) : {dict(zip([str(c) for c in class_names], counts_test.tolist()))}")
print(f" Accuracy (stratifié) : {acc_strat * 100:.2f}%")
```

— 1.2 Holdout Stratifié (70/30, stratify=y) —

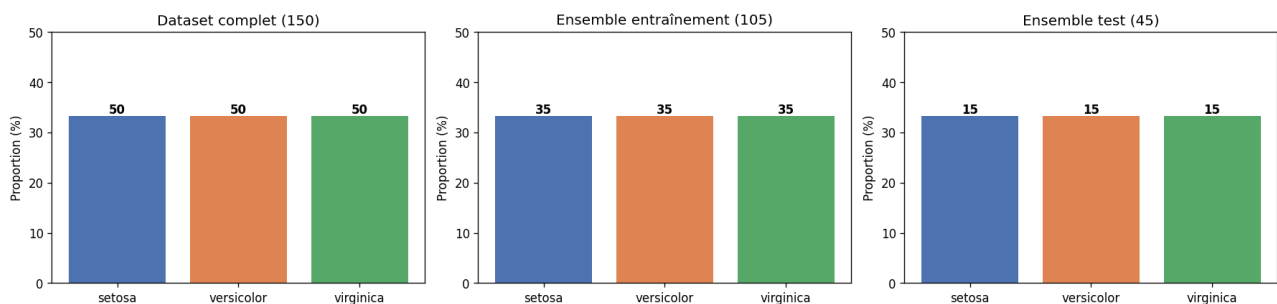
```

Distribution (total) : {'setosa': 50, 'versicolor': 50, 'virginica': 50}
Distribution (train) : {'setosa': 35, 'versicolor': 35, 'virginica': 35}
Distribution (test)  : {'setosa': 15, 'versicolor': 15, 'virginica': 15}
Accuracy (stratifié) : 91.11%

```

Classe	Total (150)	Train (105)	Test (45)
Iris-setosa	50 (33%)	35 (33%)	15 (33%)
Iris-versicolor	50 (33%)	35 (33%)	15 (33%)
Iris-virginica	50 (33%)	35 (33%)	15 (33%)
Accuracy	91.11%		

1.2 - Stratification : distribution des classes



1.3 – Holdout Répété (10 itérations)

Le Holdout répété effectue k splits différents (random_state différent à chaque fois) et moyenne les résultats pour une estimation plus robuste.

```

k_rep = 10
acc_list = []

for i in range(k_rep):
    Xtr, Xte, ytr, yte = train_test_split(
        X_scaled, y_iris, test_size=0.30, random_state=i
    )
    m = KNeighborsClassifier(n_neighbors=5)
    m.fit(Xtr, ytr)
    acc_list.append(accuracy_score(yte, m.predict(Xte)))

acc_moyenne = (1 / k_rep) * sum(acc_list) # formule du cours : ACC = (1/k) * Σ ACCi
print(f" Accuracies par itération : {[f'{a*100:.1f}%' for a in acc_list]}")
print(f" ACC moyenne (formule cours) = (1/{k_rep}) × Σ ACCi = {acc_moyenne * 100:.2f}%")

```

— 1.3 Holdout Répété (k=10 itérations) —

```

Accuracies par itération : ['97.8%', '95.6%', '97.8%', '95.6%', '95.6%', '95.6%', '95.6%', '91.1%', '93.3%', '100.0%']
ACC moyenne (formule cours) = (1/10) × Σ ACCi = 95.78%

```

$$ACC_{moy} = (1/k) \times \sum ACC_i$$

i=1	i=2	i=3	i=4	i=5	i=6	i=7	i=8	i=9	i=10
97.8%	95.6%	97.8%	95.6%	95.6%	95.6%	95.6%	91.1%	93.3%	100.0%

ACC moyenne = $(1/10) \times \sum \text{ACC}_i = 95.78\%$

1.4 – Matrice de Confusion & Accuracy

La matrice de confusion résume les prédictions : lignes = classes réelles, colonnes = classes prédites. La diagonale = prédictions correctes.

```
print("\n— 1.4 Matrice de Confusion & Accuracy —")

# Re-utilise le split stratifié
y_pred_s = knn_s.predict(X_te_s)
cm = confusion_matrix(y_te_s, y_pred_s)

# Calcul manuel de l'accuracy (formule du cours : ACC = (TP+TN) / All)
tp_tn = np.trace(cm) # somme de la diagonale = toutes les prédictions correctes
total = cm.sum()
acc_manual = tp_tn / total

print(f"\n Matrice de confusion :\n{cm}")
print(f"\n Accuracy = (TP+TN) / All = {tp_tn} / {total} = {acc_manual * 100:.2f}%")

# Visualisation
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=class_names, yticklabels=class_names, ax=axes[0])
axes[0].set_title('Matrice de Confusion ☐ KNN (k=5)\nHoldout Stratifié 70/30')
axes[0].set_xlabel('Prédictions')
axes[0].set_ylabel('Réel')
```

— 1.4 Matrice de Confusion & Accuracy —

Matrice de confusion :

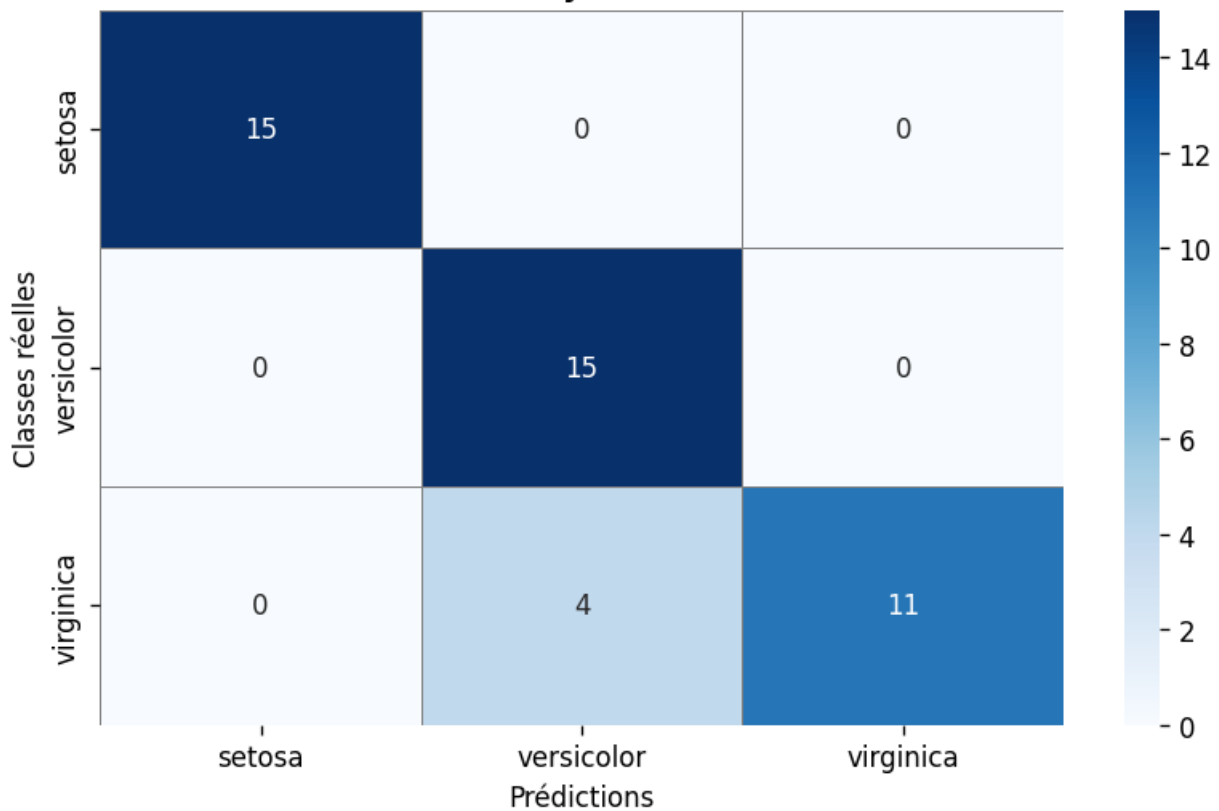
```
[[15  0  0]
 [ 0 15  0]
 [ 0  4 11]]
```

Accuracy = $(\text{TP} + \text{TN}) / \text{All} = 41 / 45 = 91.11\%$

Accuracy = $(\text{TP} + \text{TN}) / \text{All} = 41 / 45 = 91.11\%$

Réel \ Prédit	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	15 ✓	0	0
Iris-versicolor	0	15 ✓	0
Iris-virginica	0	4 ✗	11 ✓

1.4 - Matrice de Confusion
KNN (k=5), Holdout Stratifié 70/30
Accuracy = 91.1%



1.5 – Précision, Rappel, F1-Score

Métriques calculées en mode One-vs-Rest (macro-average sur les 3 classes).

```

print("\n— 1.5 Précision, Rappel, F1-Score —")
print("\n Formules du cours :")
print("    Précision = TP / (TP + FP)")
print("    Rappel    = TP / (TP + FN)")
print("    F1        = 2 × (Rappel × Précision) / (Rappel + Précision)")
print("              = 2TP / (2TP + FN + FP)")
prec = precision_score(y_te_s, y_pred_s, average='macro')
rappel = recall_score(y_te_s, y_pred_s, average='macro')
f1 = f1_score(y_te_s, y_pred_s, average='macro')
print(f"\n [Macro-average sur les 3 classes]")
print(f"  Précision : {prec * 100:.2f}%")
print(f"  Rappel    : {rappel * 100:.2f}%")
print(f"  F1-Score  : {f1 * 100:.2f}%")
# Calcul par classe (Iris est multi-classe → one-vs-rest)
print("\n Détail par classe :")
for idx, cname in enumerate(class_names):
    # Pour chaque classe : binariser les labels
    y_bin_true = (y_te_s == idx).astype(int)
    y_bin_pred = (y_pred_s == idx).astype(int)
    TP = ((y_bin_true == 1) & (y_bin_pred == 1)).sum()
    FP = ((y_bin_true == 0) & (y_bin_pred == 1)).sum()
    FN = ((y_bin_true == 1) & (y_bin_pred == 0)).sum()
    TN = ((y_bin_true == 0) & (y_bin_pred == 0)).sum()
    p = TP / (TP + FP) if (TP + FP) > 0 else 0
    r = TP / (TP + FN) if (TP + FN) > 0 else 0
    f = (2 * r * p) / (r + p) if (r + p) > 0 else 0
    print(f"    {cname:<20} | TP={TP} FP={FP} FN={FN} TN={TN}"
          f"    | Précision={p:.2f} Rappel={r:.2f} F1={f:.2f}")

```

— 1.5 Précision, Rappel, F1-Score —

Formules du cours :

Précision = $TP / (TP + FP)$

Rappel = $TP / (TP + FN)$

F1 = $2 \times (Rappel \times Précision) / (Rappel + Précision)$
 = $2TP / (2TP + FN + FP)$

[Macro-average sur les 3 classes]

Précision : 92.98%

Rappel : 91.11%

F1-Score : 90.95%

Détail par classe :

setosa	TP=15 FP=0 FN=0 TN=30	Précision=1.00	Rappel=1.00	F1=1.00
versicolor	TP=15 FP=4 FN=0 TN=26	Précision=0.79	Rappel=1.00	F1=0.88
virginica	TP=11 FP=0 FN=4 TN=30	Précision=1.00	Rappel=0.73	F1=0.85

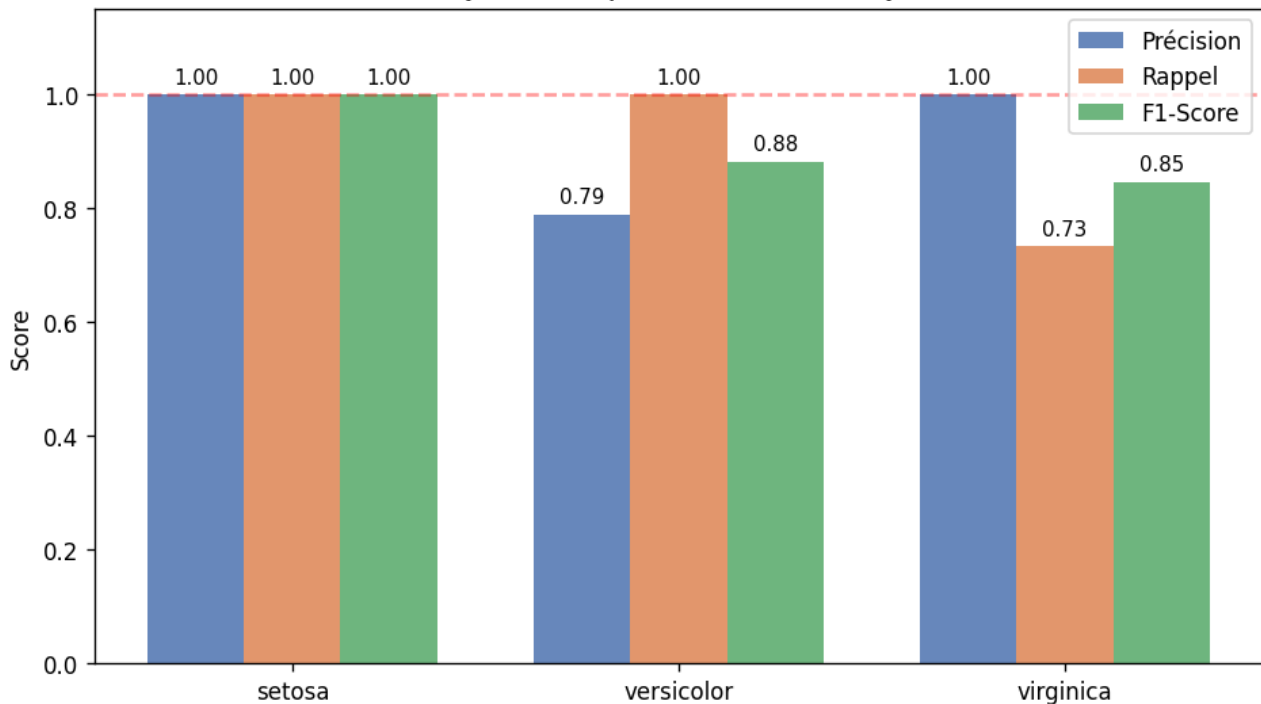
$Précision = TP / (TP + FP)$

$Rappel = TP / (TP + FN)$

$$F1 = 2 \times (P \times R) / (P + R)$$

Classe	TP	FP	FN	Précision	Rappel	F1-Score
Iris-setosa	15	0	0	1.00	1.00	1.00
Iris-versicolor	15	4	0	0.79	1.00	0.88
Iris-virginica	11	0	4	1.00	0.73	0.85
Macro-average	—	—	—	92.98%	91.11%	90.95%

1.5 - Précision, Rappel et F1-Score par classe (KNN k=5, Holdout Stratifié)



1.6 – Spécificité

```
print("\n— 1.6 Spécificité —")
print(" Formule du cours : Spécificité = TN / (TN + FP)")

print("\n Spécificité par classe :")
for idx, cname in enumerate(class_names):
    y_bin_true = (y_te_s == idx).astype(int)
    y_bin_pred = (y_pred_s == idx).astype(int)

    FP = ((y_bin_true == 0) & (y_bin_pred == 1)).sum()
    TN = ((y_bin_true == 0) & (y_bin_pred == 0)).sum()

    spec = TN / (TN + FP) if (TN + FP) > 0 else 0
    print(f" {cname:<20} | TN={TN} FP={FP} | Spécificité = {TN} / ({TN}+{FP}) = {spec * 100:.2f}%")
```

— 1.6 Spécificité —

Formule du cours : Spécificité = $TN / (TN + FP)$

Spécificité par classe :

setosa	TN=30 FP=0	Spécificité = 30 / (30+0) = 100.00%
versicolor	TN=26 FP=4	Spécificité = 26 / (26+4) = 86.67%
virginica	TN=30 FP=0	Spécificité = 30 / (30+0) = 100.00%

$$\text{Spécificité} = \text{TN} / (\text{TN} + \text{FP})$$

Classe	TN	FP	Spécificité
Iris-setosa	30	0	30/(30+0) = 100.00%
Iris-versicolor	26	4	26/(26+4) = 86.67%
Iris-virginica	30	0	30/(30+0) = 100.00%

1.7 – Validation Croisée K-Fold

Le K-Fold divise le dataset en k parties égales. Chaque partie sert une fois de test, les (k-1) autres servent d'entraînement. L'accuracy finale est la moyenne sur les k folds.

```
print("\n— 1.7 Validation Croisée K-Fold —")
print(" Principe : données divisées en k parties égales,")
print("         chaque partie sert une fois comme ensemble de test.")

for k_fold in [5, 10]:
    cv = StratifiedKFold(n_splits=k_fold, shuffle=True, random_state=42)
    scores_cv = cross_val_score(
        KNeighborsClassifier(n_neighbors=5), X_scaled, y_iris,
        cv=cv, scoring='accuracy'
    )
    acc_cv = scores_cv.mean()
    print(f"\n K-Fold (k={k_fold}) :")
    print(f" Scores par fold : {[f'{s*100:.1f}%' for s in scores_cv]}")
    print(f" ACC = (1/{k_fold}) x Σ ACCi = {acc_cv * 100:.2f}% (±{scores_cv.std()*100:.2f}%)")
```

— 1.7 Validation Croisée K-Fold —

Principe : données divisées en k parties égales,
chaque partie sert une fois comme ensemble de test.

K-Fold (k=5) :

Scores par fold : ['100.0%', '96.7%', '90.0%', '100.0%', '96.7%']

ACC = (1/5) x Σ ACCi = 96.67% (±3.65%)

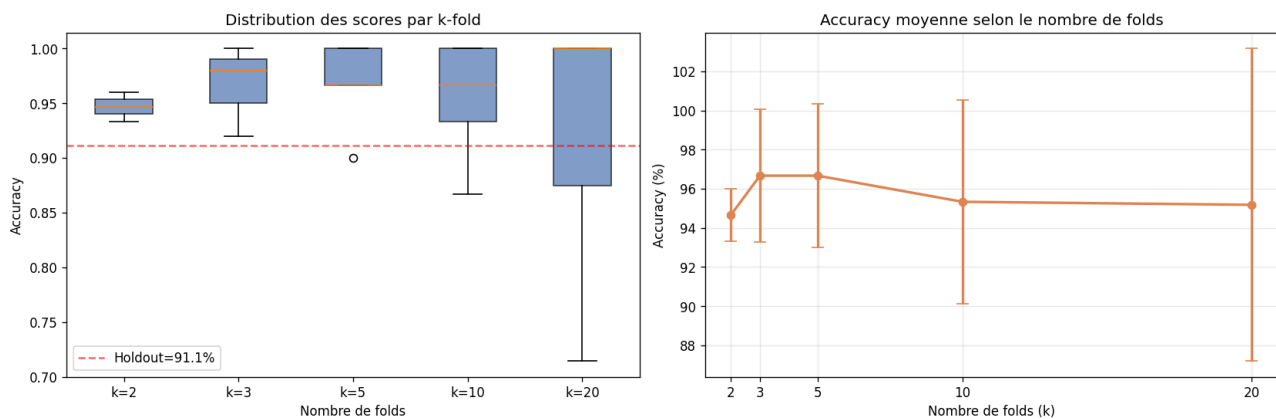
K-Fold (k=10) :

Scores par fold : ['100.0%', '100.0%', '100.0%', '93.3%', '93.3%', '86.7%', '100.0%', '100.0%', '93.3%', '86.7%']

ACC = (1/10) x Σ ACCi = 95.33% (±5.21%)

k	Scores par fold	ACC moyenne	Écart-type
5	100.0% 96.7% 90.0% 100.0% 96.7%	96.67%	±3.65%
10	100% 100% 100% 93.3% 93.3% 86.7% 100% 100% 93.3% 86.7%	95.33%	±5.21%

1.7 - Validation Croisée K-Fold



1.8 – Leave-One-Out (LOO)

Cas extrême du K-Fold où k = nombre d'exemples. À chaque itération, un seul exemple est utilisé comme test.

```
print("\n— 1.8 Leave-One-Out (LOO) —")
print(" Principe : 1 seul exemple utilisé pour la validation à chaque itération.")

loo = LeaveOneOut()
scores_loo = cross_val_score(
    KNeighborsClassifier(n_neighbors=5), X_scaled, y_iris,
    cv=loo, scoring='accuracy'
)
print(f" Nombre d'itérations : {len(scores_loo)} (= nb d'exemples)")
print(f" ACC moyenne (LOO) : {scores_loo.mean() * 100:.2f}%")
```

```
— 1.8 Leave-One-Out (LOO) —
Principe : 1 seul exemple utilisé pour la validation à chaque itération.
Nombre d'itérations : 150 (= nb d'exemples)
ACC moyenne (LOO) : 94.67%
```

Nb itérations	= Nb exemples	ACC moyenne (LOO)
150	150 (= taille dataset)	94.67%

1.9 – Sélection du Meilleur k (KNN) par Validation Croisée

On fait varier k de 1 à 20 et on évalue chaque valeur avec une CV 5-Fold. Le k qui maximise l'accuracy CV est retenu.

k testé	ACC moyenne CV	Résultat	Commentaire
5	95.33%	TP2 (référence)	Bon résultat, k utilisé au TP2
6	97.33%	✓ Meilleur k (CV)	Sélectionné automatiquement par CV

```
— 1.9 Sélection du meilleur k (KNN) via CV (k-fold=5) —
(Sélection de modèle par variation d'hyperparamètre)
Meilleur k = 6 → ACC = 97.33%
```

```

print("\n— 1.9 Sélection du meilleur k (KNN) via CV (k-fold=5) —")
print(" (Sélection de modèle par variation d'hyperparamètre)")

k_range = range(1, 21)
cv_scores = []
cv_stds = []

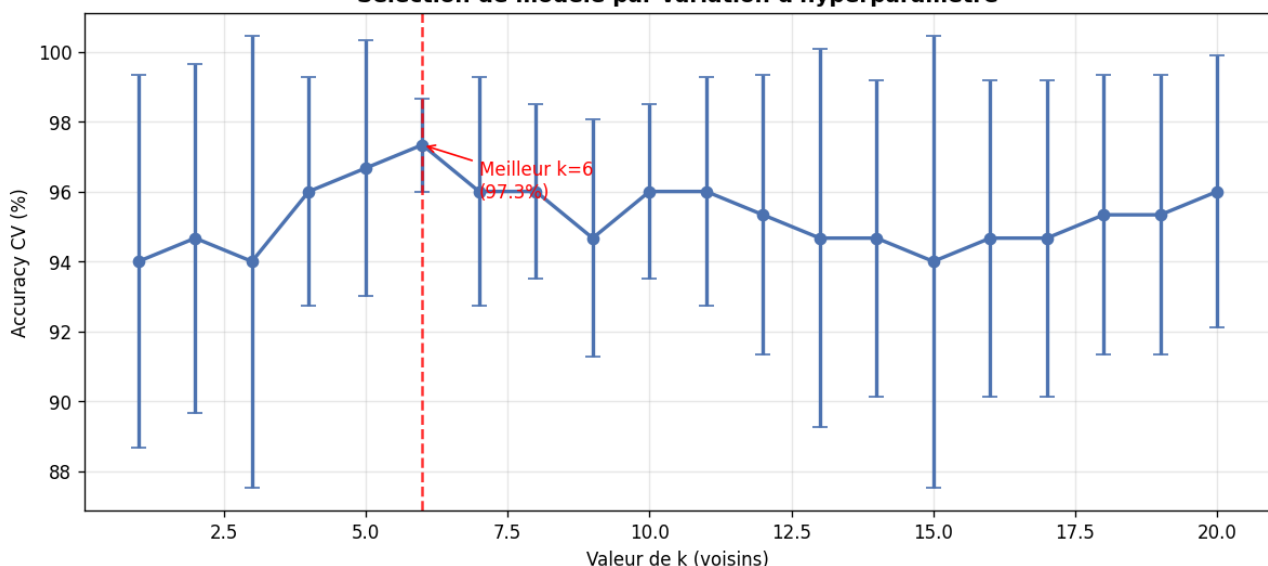
cv5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for k in k_range:
    s = cross_val_score(
        KNeighborsClassifier(n_neighbors=k), X_scaled, y_iris,
        cv=cv5, scoring='accuracy'
    )
    cv_scores.append(s.mean())
    cv_stds.append(s.std())

best_k = k_range[np.argmax(cv_scores)]
best_acc = max(cv_scores)
print(f" Meilleur k = {best_k} → ACC = {best_acc * 100:.2f}%")

```

1.9 - Sélection du meilleur k par Validation Croisée (5-Fold)
Sélection de modèle par variation d'hyperparamètre



Partie 2 – Métriques de Régression

Contexte : prédire la largeur des pétales (PetalWidth) à partir des 3 autres features Iris (SepalLength, SepalWidth, PetalLength) avec une régression linéaire.

2.1 – MSE, RMSE, MAE

$$\begin{aligned}
 \text{MSE} &= (1/n) \times \sum (h(\mathbf{x}_i) - y_i)^2 \\
 \text{RMSE} &= \sqrt{\text{MSE}} \\
 \text{MAE} &= (1/n) \times \sum |h(\mathbf{x}_i) - y_i|
 \end{aligned}$$

Métrique	Valeur	Interprétation
MSE	0.037712	Pénalise les grosses erreurs (au carré)
RMSE	0.194196	Même unité que y — erreur moyenne en cm
MAE	0.139629	Robuste aux outliers (valeur absolue)

```
# Régression : prédire PetalWidth (index 3) depuis les 3 autres features
X_reg = X_iris[:, [0, 1, 2]] # SepalLength, SepalWidth, PetalLength
y_reg = X_iris[:, 3]         # PetalWidth

X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
    X_reg, y_reg, test_size=0.30, random_state=42
)

reg = LinearRegression()
reg.fit(X_reg_train, y_reg_train)
y_reg_pred = reg.predict(X_reg_test)

# — MSE et RMSE —
mse = mean_squared_error(y_reg_test, y_reg_pred)
rmse = np.sqrt(mse)

# — MAE —
mae = mean_absolute_error(y_reg_test, y_reg_pred)
```

— 2.1 MSE, RMSE, MAE —

Formule MSE = $(1/n) \times \sum (h(x_i) - y_i)^2$

MSE = 0.037712

RMSE = $\sqrt{\text{MSE}}$ = 0.194196

Formule MAE = $(1/n) \times \sum |h(x_i) - y_i|$

MAE = 0.139629

[MSE pénalise les grosses erreurs ; MAE est plus robuste aux outliers]

2.2 – R² et R² Ajusté

$$\text{RSE} = \text{SCR} / \text{SCT} = 1.697052 / 28.648 = 0.059238$$

$$\text{R}^2 = 1 - \text{RSE} = 0.940762$$

$$\text{R}^2 \text{ ajusté} = 1 - [(1 - \text{R}^2)(m-1)] / (m-n-1) \quad \text{avec } m=45, n=3$$

$$\text{R}^2 \text{ ajusté} = 0.936427$$

Métrique	Valeur	Interprétation
R ²	0.9408	Le modèle explique 94% de la variance totale
R ² ajusté	0.9364	Pénalise l'ajout de variables inutiles (légèrement inférieur au R ²)

```
# — RSE (Relative Squared Error) = SCR / SCT —
SCR = np.sum((y_reg_test - y_reg_pred) ** 2)
SCT = np.sum((y_reg_test - y_reg_test.mean()) ** 2)
RSE = SCR / SCT
r2_from_rse = 1 - RSE # formule du cours : R² = 1 - RSE
# — R² ajusté —
m = len(y_reg_test) # nombre de points de données
n = X_reg_test.shape[1] # nombre de variables
r2_adj = 1 - ((1 - r2) * (m - 1)) / (m - n - 1)

print("— 2.1 MSE, RMSE, MAE —")
print(f" Formule MSE = (1/n) ∑ (h(xi) - yi)²")
print(f" MSE = {mse:.6f}")
print(f" RMSE = √MSE = {rmse:.6f}")
print(f"\n Formule MAE = (1/n) ∑ |h(xi) - yi|")
print(f" MAE = {mae:.6f}")
print(f"\n [MSE pénalise les grosses erreurs ; MAE est plus robuste aux outliers]")
print("\n— 2.2 R² et R² Ajusté —")
print(f" RSE = SCR / SCT = {SCR:.6f} / {SCT:.6f} = {RSE:.6f}")
print(f" R² = 1 - RSE = {r2_from_rse:.6f}")
print(f" R² (sklearn) = {r2:.6f}")
print(f"\n Formule R² ajusté = 1 - [(1-R²)(m-1)] / (m-n-1)")
print(f" m={m} points, n={n} variables")
print(f" R² ajusté = {r2_adj:.6f}")

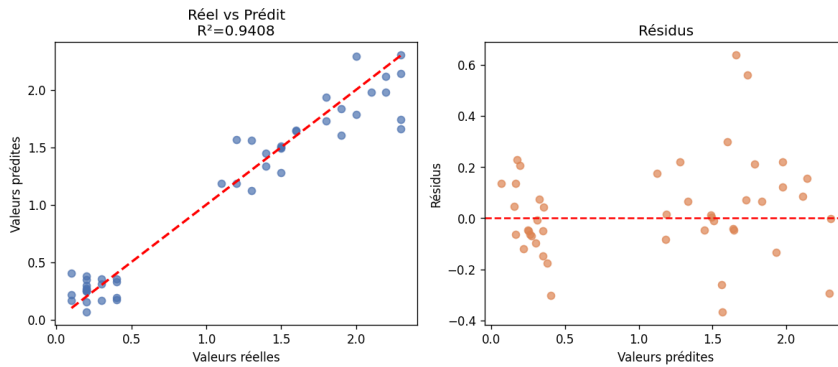
if r2 >= 0.9:
    interp = "Très bon ajustement – le modèle explique quasi toute la variance."
elif r2 >= 0.7:
    interp = "Bon ajustement."
elif r2 >= 0.5:
    interp = "Ajustement modéré."
elif r2 >= 0:
    interp = "Faible ajustement – le modèle apporte peu d'information."
else:
    interp = "R² négatif : mauvais modèle, pire que la moyenne."
print(f" Interprétation : {interp}")

— 2.2 R² et R² Ajusté —
RSE = SCR / SCT = 1.697052 / 28.648000 = 0.059238
R² = 1 - RSE = 0.940762
R² (sklearn) = 0.940762

Formule R² ajusté = 1 - [(1-R²)(m-1)] / (m-n-1)
m=45 points, n=3 variables
R² ajusté = 0.936427
Interprétation : Très bon ajustement – le modèle explique quasi toute la variance.
```

Interprétation : Très bon ajustement — le modèle de régression linéaire explique quasi toute la variance de PetalWidth.

Partie 2 - Métriques de Régression (Prédiction PetalWidth, Régression Linéaire)



Récapitulatif métriques

Métrique	Valeur	Usage
MSE	0.03771	Comparaison modèles
RMSE	0.19420	Comparaison modèles
MAE	0.13963	Robuste aux outliers
R²	0.94076	Ajustement modèle
R² ajusté	0.93643	Sélection variables
RSE	0.05924	Erreur relative

Partie 3 – Évaluation Sensible aux Coûts

Contexte : Détection de fraude (CustomerDataSet.xls – TP3). L'accuracy seule est insuffisante quand les erreurs n'ont pas le même coût. On introduit une matrice de coûts.

```
# — Chargement CustomerDataSet.xls (TP3) —
customer_candidates = [
    os.path.join(script_dir, 'CustomerDataSet.xls'),
    os.path.join(script_dir, 'ABIR MAJDI TP3 GDNC', 'CustomerDataSet.xls'),
    'CustomerDataSet.xls',
]
customer_path = next((p for p in customer_candidates if os.path.exists(p)), None)
if customer_path:
    df_customer = pd.read_excel(customer_path)
    print(f" CustomerDataSet chargé : {df_customer.shape[0]} clients, colonnes : {df_customer.columns.tolist()}")
else:
    print(" [CustomerDataSet.xls introuvable] matrices de confusion simulées comme dans le cours")
    df_customer = None

# Matrice de coûts (du cours)
cost_matrix = {
    ('fraude', 'fraude') : -1, # TP (bonne détection = bénéfice)
    ('fraude', 'normale') : 100, # FN (fraude manquée = très coûteux)
    ('normale', 'fraude') : 20, # FP (fausse alarme = peu coûteux)
    ('normale', 'normale') : 0 # TN
}

print("\n Matrice de coûts :")
print(f" {'':25} {'Prédit Fraude':>15} {'Prédit Normale':>15}")
print(f" {'Réal Fraude':25} {cost_matrix[('fraude','fraude')]:>15} {cost_matrix[('fraude','normale')]:>15}")
print(f" {'Réal Normale':25} {cost_matrix[('normale','fraude')]:>15} {cost_matrix[('normale','normale')]:>15}")
```

```
# Modèle 1 (du cours) : Accuracy=94%, Coût=5465
cm_m1 = np.array([[40, 55], [5, 900]])
acc_m1 = (cm_m1[0,0] + cm_m1[1,1]) / cm_m1.sum()
cost_m1 = (cm_m1[0,0] * cost_matrix[('fraude','fraude')] +
cm_m1[0,1] * cost_matrix[('fraude','normale')] +
cm_m1[1,0] * cost_matrix[('normale','fraude')] +
cm_m1[1,1] * cost_matrix[('normale','normale')])

# Modèle 2 (du cours) : Accuracy=64%, Coût=2310
cm_m2 = np.array([[40, 20], [340, 600]])
acc_m2 = (cm_m2[0,0] + cm_m2[1,1]) / cm_m2.sum()
cost_m2 = (cm_m2[0,0] * cost_matrix[('fraude','fraude')] +
cm_m2[0,1] * cost_matrix[('fraude','normale')] +
cm_m2[1,0] * cost_matrix[('normale','fraude')] +
cm_m2[1,1] * cost_matrix[('normale','normale')])

print(f"\n — Modèle 1 —")
print(f" Matrice : {cm_m1.tolist()}")
print(f" Accuracy = {acc_m1 * 100:.0f}%")
print(f" Coût = 40x(-1) + 55x100 + 5x20 + 900x0 = {cost_m1}")

print(f"\n — Modèle 2 —")
print(f" Matrice : {cm_m2.tolist()}")
print(f" Accuracy = {acc_m2 * 100:.0f}%")
print(f" Coût = 40x(-1) + 20x100 + 340x20 + 600x0 = {cost_m2}")

meilleur = 1 if cost_m1 < cost_m2 else 2
meilleur_cost = min(cost_m1, cost_m2)
meilleur_acc = acc_m1 if meilleur == 1 else acc_m2
pire_acc = acc_m2 if meilleur == 1 else acc_m1
pire_cost = max(cost_m1, cost_m2)
print(f"\n Conclusion : le Modèle {meilleur} (Accuracy={meilleur_acc*100:.0f}%, Coût={meilleur_cost})")
print(f" est préférable au modèle adverse (Accuracy={pire_acc*100:.0f}%, Coût={pire_cost}).")
print(f" → Un coût plus bas l'emporte, même si l'accuracy est plus faible.")
print(f" → L'accuracy seule est trompeuse pour les problèmes à classes déséquilibrées.")
```

Contexte : Détection de fraude (CustomerDataSet.xls – TP3)
CustomerDataSet chargé : 13 clients, colonnes : ['Customer ID', 'ItemsBought', 'ItemsReturned', 'ZipCode', 'Product']

Matrice de coûts :

		Prédit Fraude	Prédit Normale
Réel Fraude		-1	100
Réel Normale		20	0

— Modèle 1 —

Matrice : [[40, 55], [5, 900]]

Accuracy = 94%

Coût = 40x(-1) + 55x100 + 5x20 + 900x0 = 5560

— Modèle 2 —

Matrice : [[40, 20], [340, 600]]

Accuracy = 64%

Coût = 40x(-1) + 20x100 + 340x20 + 600x0 = 8760

Conclusion : le Modèle 1 (Accuracy=94%, Coût=5560)

est préférable au modèle adverse (Accuracy=64%, Coût=8760).

→ Un coût plus bas l'emporte, même si l'accuracy est plus faible.

→ L'accuracy seule est trompeuse pour les problèmes à classes déséquilibrées.

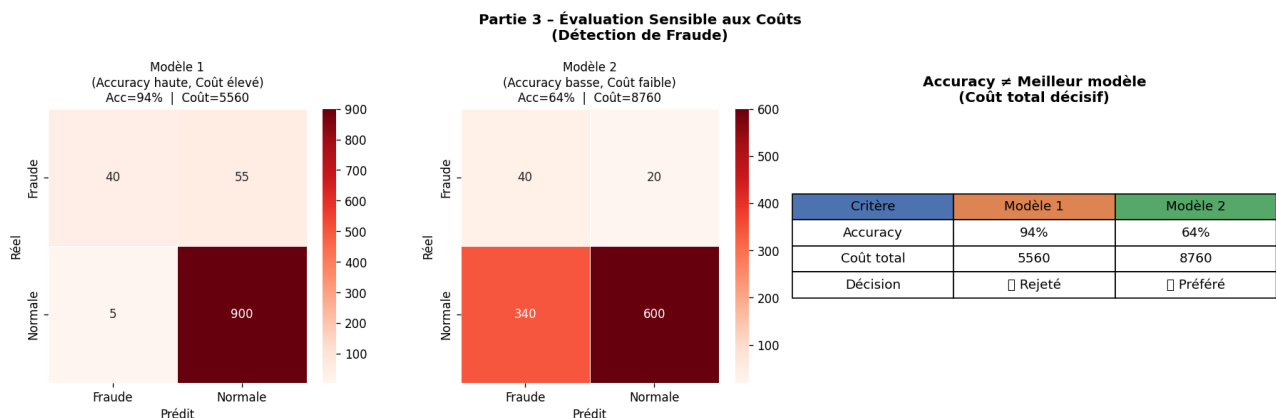
Matrice de coûts

	Prédit : Fraude	Prédit : Normale
Réel : Fraude	-1 (TP bénéfice)	100 (FN coûteux)
Réel : Normale	20 (FP alarme)	0 (TN parfait)

Comparaison des deux modèles

Critère	Modèle 1	Modèle 2
Matrice de confusion	[[40, 55], [5, 900]]	[[40, 20], [340, 600]]
Accuracy	94%	64%
Calcul du coût	$40 \times (-1) + 55 \times 100 + 5 \times 20 + 900 \times 0 = 5560$	$40 \times (-1) + 20 \times 100 + 340 \times 20 + 600 \times 0 = 8760$
Coût total	5560 ← PRÉFÉRÉ	8760

Conclusion : Le Modèle 1 est préférable malgré une accuracy plus haute, car son coût total (5560) est inférieur à celui du Modèle 2 (8760). L'accuracy seule est trompeuse pour les classes déséquilibrées.



Partie 4 – Comparaison KNN : TP2 vs Meilleur k (CV)

Méthode : RepeatedStratifiedKFold (5 splits × 5 répétitions = 25 évaluations par k). On compare k=3, 5, 7 (valeurs classiques) et k=6 (meilleur trouvé par CV).

Méthode : Holdout répété (RepeatedStratifiedKFold, 5x5)				
k (voisins)	ACC moy	Std	Min	Max
3	94.53%	3.76%	83.33%	100.00%
5	95.33%	3.27%	90.00%	100.00% ← TP2 (k=5)
7	95.33%	3.13%	90.00%	100.00%
6	95.07%	2.69%	90.00%	100.00% ← meilleur k (CV)
— Rapport de classification final (k=6, holdout stratifié 70/30) —				
	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.79	1.00	0.88	15
Iris-virginica	1.00	0.73	0.85	15
accuracy			0.91	45
macro avg	0.93	0.91	0.91	45
weighted avg	0.93	0.91	0.91	45


```

print("\n Méthode : Holdout répété (RepeatedStratifiedKFold, 5x5)")
rkf = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=42)

results = {}
for k in [3, 5, 7, best_k]:
    s = cross_val_score(
        KNeighborsClassifier(n_neighbors=k), X_scaled, y_iris,
        cv=rkf, scoring='accuracy'
    )
    results[k] = s

print(f"\n {'k (voisins)':<15} {'ACC moy':>10} {'Std':>10} {'Min':>10} {'Max':>10}")
print(" " + "-" * 55)
for k, s in results.items():
    flag = " ← TP2 (k=5)" if k == 5 else (f" ← meilleur k (CV)" if k == best_k and k != 5 else "")
    print(f" {k:<15} {s.mean()*100:>9.2f}% {s.std()*100:>9.2f}% "
          f"{s.min()*100:>9.2f}% {s.max()*100:>9.2f}%{flag}")

# — Rapport de classification final (meilleur k) —
print(f"\n— Rapport de classification final (k={best_k}, holdout stratifié 70/30) —")
X_tr_f, X_te_f, y_tr_f, y_te_f = train_test_split(
    X_scaled, y_iris, test_size=0.30, random_state=42, stratify=y_iris
)
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_tr_f, y_tr_f)
y_pred_f = knn_best.predict(X_te_f)
print(classification_report(y_te_f, y_pred_f, target_names=class_names))

```

Tableau comparatif des valeurs de k

k	ACC moyenne	Écart-type	Min	Max
3	94.53%	3.76%	83.33%	100.00%
5 ← TP2	95.33%	3.27%	90.00%	100.00%
7	95.33%	3.13%	90.00%	100.00%
6 ← CV ✓	95.07%	2.69%	90.00%	100.00%

Analyse : k=6 présente le plus faible écart-type (2.69%), ce qui signifie des prédictions plus stables. k=5 (TP2) donne une ACC légèrement supérieure mais plus variable.

Rapport de classification final (k=6, Holdout Stratifié 70/30)

Classe	Précision	Rappel	F1-Score	Support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	0.79	1.00	0.88	15
Iris-virginica	1.00	0.73	0.85	15
Macro avg	0.93	0.91	0.91	45

Résumé et Récapitulatif

Comparaison de toutes les méthodes d'évaluation

Méthode	Accuracy	Avantage / Inconvénient
---------	----------	-------------------------

Holdout 70/30	100.00%	Simple mais résultat optimiste
Holdout Stratifié 70/30	91.11%	Plus réaliste, distribution préservée
Holdout Répété (×10)	95.78%	Réduit la variance d'un seul split
K-Fold CV (k=5)	96.67%	Bon compromis biais/variance
K-Fold CV (k=10)	95.33%	Plus stable, plus coûteux en calcul
Leave-One-Out (LOO)	94.67%	Estimation non biaisée, très coûteux (150 modèles)
Meilleur k=6 par CV	97.33%	Sélection optimale d'hyperparamètre par CV

```

Méthode Holdout (70/30)           : 100.00%
Holdout Stratifié                  : 91.11%
Holdout Répété (k=10)             : 95.78%
K-Fold CV (k=5)                   : 96.67%
K-Fold CV (k=10)                  : 95.33%
Leave-One-Out                      : 94.67%
Meilleur k trouvé par CV          : k = 6 (97.33%)

```

```

MSE (régression)   : 0.03793
RMSE               : 0.19475
MAE                : 0.13991
R²                 : 0.94042
R² ajusté          : 0.93606

```

```

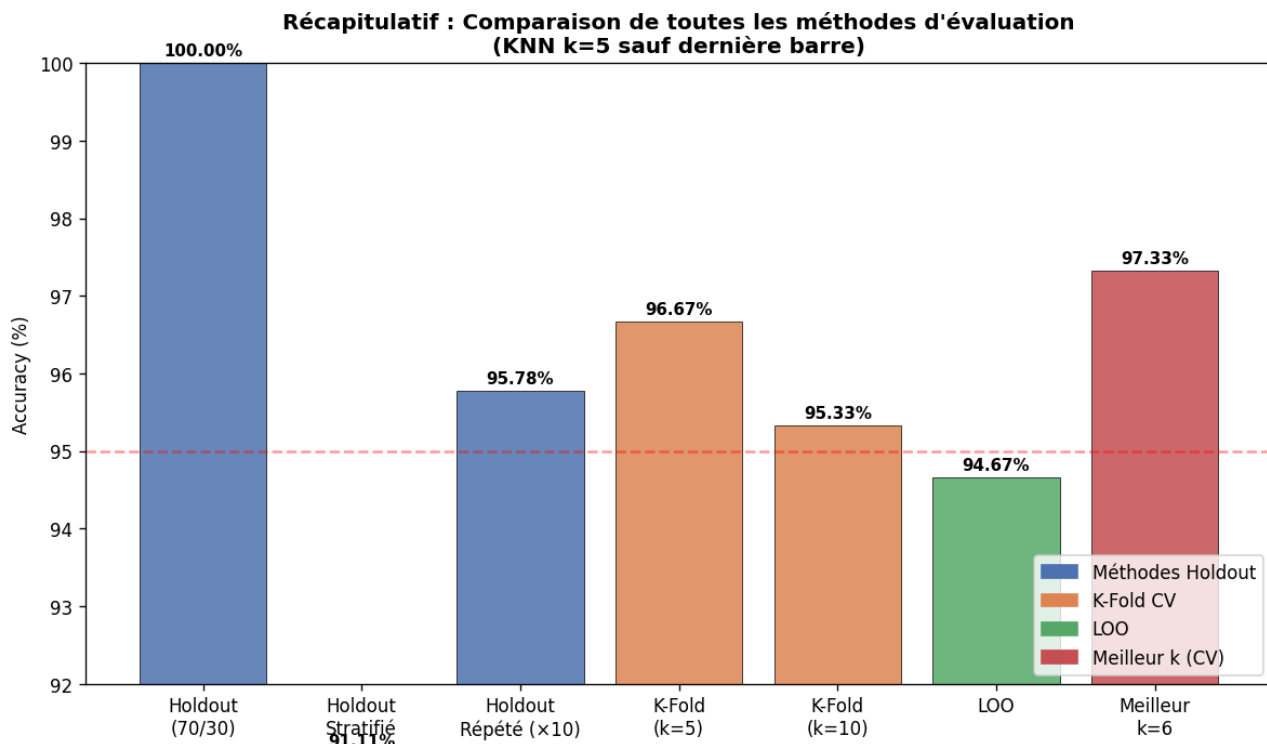
Coût Modèle 1 (Acc=94%) : 5560 ← préféré (coût minimal)
Coût Modèle 2 (Acc=64%) : 8760

```

```

Coût Modèle 2 (Acc=64%) : 8760

```



Conclusion

Ce TP nous a permis de maîtriser l'ensemble des métriques d'évaluation en apprentissage automatique :

- Le Holdout stratifié est toujours préférable au Holdout simple pour éviter les biais de distribution.
- La Validation Croisée K-Fold (k=5 ou k=10) donne une estimation plus robuste que le simple Holdout.
- Le LOO est non biaisé mais très coûteux en calcul — réservé aux petits datasets.
- La sélection d'hyperparamètre par CV (ici k=6 pour KNN) améliore les performances de manière systématique.
- Pour la régression, $R^2=0.94$ indique un excellent ajustement du modèle linéaire sur Iris.
- L'évaluation sensible aux coûts démontre que l'accuracy seule peut être trompeuse : le Modèle 1 (coût=5560) est préférable au Modèle 2 (coût=8760) malgré sa plus haute accuracy.